Jordan Forbes
ECE-GY 9953 Gen AI Based Chip Design
10/7/26

2-to-1 Mux

Golden Model: The golden model takes 3 arguments, a, b, sel. The LLMs binary-string patterns are converted into ints before the call. It computes y = a if sel == 0. This was generated from the description, not copied from the Verilog. It returns a {'y':y} dictionary.

Pattern File: A JSON list with one dictionary per test. There are 8 here and all are 2^3 combinations. Each dictionary holds the inputs a, b, and sel as ints plus expected_outputs, a nested dictionary such as {"y": 1} from the golden model.

Updater Effect: For the initial testbench only the Inputs are applied and printed, with no pass/fail. The Final testbench made a few changes. It declared 3 counters (passed, failed, and total) at module level and initialized them to 0. It also added a comparison after every test case that prints both the actual and expected value of y. It used the case-equality operator “===” instead of “==”. It then added a summary of total passed and failed tests. The final testbench passed all 8 tests.

4-bit adder

Golden Model: The function takes 2 inputs a and b as integers. It first masks both inputs to 4 bits, then adds the 2 inputs as full-precision integers. Sum is the lower 4 bits of the result. Carry is 1 when the result is greater than 15. The model returns {'sum': sum_result, 'carry': carry}

Pattern File: The file is a list of 10 entries. Each one holds the inputs as integers and an expected_outputs dictionary with both outputs. All 10 entries have expected outputs and none received an error. The patterns include zero inputs, the largest sum (15+15=30) and the smallest sum that produces a carry (14+2=16).

Updater Effect: In the initial testbench, each test only assigns the inputs, waits 10ns and prints a, b, sum, and carry. Nothing Compares the outputs to expected values. In the final testbench it declared 2 counters to count passed and failed tests, added a comparison after each test, using “===”, so an x or z output counts as a failure. A test passes only if both outputs match, but on a failure sum and carry are reported separately with the expected and actual values, giving 20 checks total. It also added a summary of the total passed and failed tests.

8-to-3 Priority Encoder

I created an 8-to-3 priority encoder. It is purely combinational but its corner cases are easy to get wrong: Several request bits are set at once, all bits set and no bits set. The natural language description given specified the port and bit widths, the design is purely combinational and the corner cases to test: all zeros, all ones, every one-hot value, bit 0 combined with one higher bit, and 00000001 against 00000000. The most important prompt choice was the explicit test requirement at the end of the description. The count and the corner-case list had to go in the specification itself.

I introduced a bug into my priority encoder. The casez branch for 6 bit was changed from idx = 3'd6 to idx = 3'd5.Whenever bit 6 was the highest set bit of req, the encoder reported index 5 instead of 6. The buggy design was saved in a separate file “priority_encoder8_buggy.v” and simulated with the same testbench. As expected 3 of the tests failed. After fixing the DUT all of the test cases passed.